

```

"""
sentiment_utils.py
-----
Separate from gemini_utils.py on purpose,
so gemini_utils.py itself
doesn't need to be touched for this
feature.

analyze_sentiment() – a silent, structured-
JSON call (the lead never
sees it) that scores how "hot"/interested a
lead sounds RIGHT NOW,
based on their latest message + a bit of
recent history. Used so the
dashboard can surface the most promising
leads first.

Reuses the already-configured Gemini
client, retry logic, and model
name from gemini_utils.py instead of
duplicating any of that here.
"""

import json
import re

import gemini_utils # reuses _client,
_generate_with_retry, MODEL_NAME

```

```
SENTIMENT_PROMPT_TEMPLATE = """Analyze this
WhatsApp message from a real-estate lead to
gauge how
interested/ready-to-buy they sound RIGHT
NOW, based only on tone and urgency in this
message
(and briefly the recent conversation if
given).
```

```
Return ONLY a valid JSON object, nothing
else – no explanation, no markdown fences.
```

```
Fields:
```

```
- interest_level: one of "hot", "warm",
"cold"
  - "hot": ready to move fast – asking
about price/visit/booking, urgent tone,
decisive language
  - "warm": genuinely interested but still
deciding, comparing, or has open questions
  - "cold": just browsing, vague, non-
committal, or early/generic questions
- reason: a short phrase (max 6 words)
explaining why, in English (e.g. "asked for
site visit directly")
```

```
Recent conversation (for context only, may
be empty):
{history}
```

```
Latest message: "{message}"
```

```
JSON: """
```

```

def analyze_sentiment(user_message: str,
history: list = None) -> dict:
    """
        Returns {"interest_level":
"hot"|"warm"|"cold", "reason": str or
None}.
        Defaults to "warm" with no reason if
anything goes wrong – a neutral
guess is safer than crashing the main
reply flow.
    """
    default = {"interest_level": "warm",
"reason": None}
    if not gemini_utils._client or not
user_message:
        return default

    history_lines = []
    for turn in (history or [])[-6:]: #
last few turns is enough context
        speaker = "Lead" if
turn.get("role") == "user" else "Assistant"
        history_lines.append(f"{speaker}:
{turn.get('text', '')}")
    history_text = "\n".join(history_lines)
    if history_lines else "(no prior messages)"

    prompt =
SENTIMENT_PROMPT_TEMPLATE.format(
        history=history_text,
message=user_message.replace("'", "'")
    )

```

```
try:
    response =
gemini_utils._generate_with_retry(prompt)
    text = (response.text or
    "").strip()
    text = re.sub(r"^```(json)?|```$",
    "", text, flags=re.MULTILINE).strip()
    parsed = json.loads(text)
    if parsed.get("interest_level") in
    ("hot", "warm", "cold"):
        default["interest_level"] =
    parsed["interest_level"]
    if parsed.get("reason"):
        default["reason"] =
    str(parsed["reason"][:60])
    return default
except Exception as e:
    print("[sentiment_utils] Sentiment
analysis error:", e)
    return default
```